

CS505 HW2: Transformers - Written Responses

Author: Zhengzheng Tang

BUID: U07312313

Date: February 2026

Task 1: Transformers From Scratch (41 points)

Q1 (20 points) - Single-Head Attention

I implemented single-headed attention with causal masking in the `Transformer` class. The implementation includes:

- **Q, K, V projections:** Linear transformations mapping hidden states to query, key, and value matrices
- **Scaled dot-product attention:** Computing attention scores as $\frac{QK^T}{\sqrt{d_k}}$
- **Causal masking:** Using `torch.triu` to create an upper triangular mask, setting future positions to $-\infty$ before softmax
- **Output projection:** Linear transformation after attention
- **Residual connections and Layer Normalization**

Final Single-Head Transformer Dev Perplexity on Pile: 2479.7351

Q2 (5 points) - Layer Normalization

I implemented Layer Normalization as follows:

$$\hat{h} = \gamma \frac{h - \mu}{\sigma + \epsilon} + \beta$$

where μ and σ are computed along the last dimension (hidden_dim), and γ, β are learnable parameters. I used $\epsilon = 10^{-5}$ for numerical stability.

The Layer Norm is applied after both the self-attention and MLP blocks, following the residual connections.

Dev Perplexity after adding Layer Norm: 2479.7351

Q3 (10 points) - Multi-Head Attention

I extended the `Transformer` class to `MultiHeadTransformer` with 4 attention heads. The key changes:

1. **Reshape Q, K, V:** From (B, T, d_h) to $(B, \text{num_heads}, T, \text{head_dim})$
2. **Parallel attention:** Each head computes attention independently with $\text{head_dim} = d_h/n$
3. **Concatenation:** Merge all heads back to (B, T, d_h)
4. **Output projection:** Final linear transformation

Multi-Head Transformer Dev Perplexity on Pile: 1318.6891

Comparison: The multi-head attention model (perplexity 1318.69) significantly outperforms the single-head model (perplexity 2479.74). This improvement of ~47% demonstrates that multiple attention heads allow the model to attend to information from different representation subspaces at different positions, capturing more diverse patterns in the data.

Q4 (6 points) - Temperature Sampling

I implemented temperature sampling in the `generate` function:

$$y \sim \text{softmax}(e/\tau)$$

where e is the logit vector and τ is the temperature hyperparameter.

Generated outputs (Multi-Head Pile Model, temperature=0.8, max 30 tokens):

- Prompt:** "The quick brown fox"
Generated: "The quick brown fox.d)- Tess A is of We data2, the the the surface by in marriage and it and the cats talking. But a a to"
- Prompt:** "In the beginning"
Generated: "In the beginning is the mounting with the 1977 of refuted of once, mac is Mongo en).or:]. For findings.[[oly. The a evaluated generally those"
- Prompt:** "What is 5 + 5?"
Generated: "What is 5 + 5?, and Sy and a a power. In the decline community +a, alternative-in. This's Facts. We exist (1 and E],["
- Prompt:** "The meaning of life is"
Generated: "The meaning of life is blog pathway the the single SUN- (; Un Long injuries, the to 21 to be presence. In the long ATP is " Accordingly. Our any"

Observations: The model's outputs are largely incoherent and contain grammatical errors, random punctuation, and disconnected phrases. For prompt (3) "What is 5 + 5?", the model did not attempt to answer the mathematical question but instead continued with unrelated text. This is expected because: (1) the model is very small (~14M parameters), (2) it was trained on limited data (10,000 samples), and (3) language models trained only on next-token prediction don't inherently learn to follow instructions or answer questions.

Task 2: Transfer Learning via Fine-tuning (13 points)

Q5 (3 points) - Shakespeare Training Comparison

Model	Shakespeare Dev Perplexity
Pile model (zero-shot, no adaptation)	16.3631
Trained from scratch on Shakespeare	8.2058
Pile model fine-tuned on Shakespeare	7.4496

Analysis: The results match expectations:

- The zero-shot Pile model performs worst (16.36) because Shakespeare's archaic English differs significantly from modern internet text.

- Training from scratch on Shakespeare achieves good perplexity (8.21) by learning the domain-specific patterns.
- Fine-tuning the Pile model achieves the best perplexity (7.45), demonstrating that pre-training provides useful general language knowledge that transfers well to the new domain, even with a lower learning rate and fewer epochs.

Q6 (2 points) - Catastrophic Forgetting

Fine-tuned model perplexity on Pile dev: 11083.8555

Original Pile model perplexity on Pile dev: 1318.6891

Analysis: The fine-tuned model's perplexity on Pile increased dramatically from 1318.69 to 11083.86 (an 8.4x increase). This phenomenon is known as **catastrophic forgetting**. When the model is fine-tuned on Shakespeare data, it adjusts its weights to better predict Shakespeare-style text, which causes it to "forget" the patterns it learned from the Pile. The model's parameters shift towards the new distribution, degrading performance on the original domain.

Q7 - GPT-2 Fine-tuning

Q7a (3 points) - GPT-2 Perplexity

GPT-2 (Shakespeare fine-tuned) Dev Perplexity: 85.6376

Best Hyperparameters:

- Learning rate: 5e-4
- Number of epochs: 3
- Batch size: 4

Q7b (5 points) - Generation Comparison (16 total outputs)

GPT-2 Fine-tuned Generations (8 outputs):

Shakespeare Prompts:

1. **"To be or not to be"**
"To be or not to be. Therefore we'll issue out again, and again. What say you, sir? If any of your own choice, 'tis wit in request; For we have need of good cheer: 'tis fit for our purpose; My noble uncle is the"
2. **"O Romeo, Romeo"**
"O Romeo, Romeo, Romeo! thou hast no cause to fear. What comfort is this To thee that with the help of Mercutio? I am Angelo, and thyself preserve! If thou refuse, let me know thee will not; For, as thou hast"
3. **"Friends, Romans, countrymen"**
"Friends, Romans, countrymen, and all, and one amongst you. Fare you well. You know our enemy's house, our countrymen, Of whom you and yours are almost at odds; Neighbour Tybalt, our king, Is the house of Antium;"
4. **"All the world's a stage"**
"All the world's a stage! I have a stomach; but 'tis my stomach. Shall we stand to such an officer? Must we be plucked down and thrown away? What say'st thou, my sweet son? Thy last revenge for this offence? Let him be"

General Prompts:

5. **"The quick brown fox"**

"The quick brown fox: The king Henry the Sixth Was but a messenger to the people. Our great-bred horses, that were with the queen, Came to the Duke of Norfolk in arms; His noble blood, that collected The noble commons' blood, that collected The"

6. **"In the beginning"**

"In the beginning: Good Grumio, what hast thou done there? 'Tis done, and we'll do thee justice. Hast thou no clients? no? no? no? go to. What hast thou done there? go to the world; or"

7. **"What is 5 + 5?"**

"What is 5 + 5? what news? where's the Earl of Warwick? What is Rice, the Earl of Wiltshire? What news? His regiment is dispersed by tempest; And they are dispersed, with the rest, Both with the king and the new"

8. **"The meaning of life is"**

"The meaning of life is to make a puppet of him. O, the meaning of life! he is a fool, by nature, by nature, to put forth; and, since he lives, he's a fool. The meaning of my life, my meaning, is"

Custom Transformer (Fine-tuned on Shakespeare) Generations (8 outputs):

Shakespeare Prompts:

1. **"To be or not to be"**

"To be or not to be you?"

2. **"O Romeo, Romeo"**

"O Romeo, Romeo: Merc: O's a fear:"

3. **"Friends, Romans, countrymen"**

"Friends, Romans, countrymen, an we be same he, that And at. I ambassadors to tim in talking are wife what thy."

4. **"All the world's a stage"**

"All the world's a stage."

General Prompts:

5. **"The quick brown fox"**

"The quick brown fox: Upon that no word.; be eyes."

6. **"In the beginning"**

"In the beginning you."

7. **"What is 5 + 5?"**

"What is 5 + 5? What And he, say your-"

8. **"The meaning of life is"**

"The meaning of life is bloody as,:"

Why GPT-2's outputs are better: GPT-2 was pre-trained on ~40GB of internet text (WebText), giving it a much stronger foundation of English grammar, vocabulary, and coherent sentence structure. Even though our custom model was fine-tuned on Shakespeare, it only had 14M parameters and limited pre-training data (10K samples), making it unable to generate fluent text.

Why GPT-2 has worse perplexity but better generation: This apparent paradox occurs because:

1. **Perplexity measures average prediction accuracy** - our small custom model may overfit to Shakespeare's specific token patterns, achieving lower perplexity on held-out Shakespeare data.
2. **Generation quality requires broader knowledge** - GPT-2's extensive pre-training gives it better understanding of grammar, semantics, and coherent narrative structure, even if it assigns slightly lower probability to some Shakespeare-specific word choices.

Task 3: Conceptual Questions (11 points)

Q8 (3 points) - Positional Embeddings

Why do Transformers need positional embeddings while RNNs don't?

RNNs process sequences sequentially, maintaining a hidden state that implicitly encodes position through the order of processing. Each token is processed one after another, so the model inherently knows which token came first.

Transformers, in contrast, process all tokens in parallel using self-attention, which computes pairwise relationships between all positions simultaneously. Without positional embeddings, the attention mechanism is permutation-invariant—it cannot distinguish between "the cat sat on the mat" and "mat the on sat cat the." Positional embeddings inject position information into the token representations, allowing the model to understand sequential order.

Q9 (2 points) - Residual Connections and Overfitting

Would removing residual connections help with overfitting?

No, removing residual connections would likely **not** improve performance and could make things worse. While simplifying the model might seem like a solution for overfitting, residual connections serve a critical purpose: they enable gradient flow through deep networks. Without them:

1. Gradients may vanish during backpropagation, making training unstable or impossible
2. The model may become harder to optimize, potentially getting stuck in poor local minima

Better approaches to address overfitting include: dropout, weight decay, data augmentation, early stopping, or reducing model size (fewer layers/dimensions).

Q10 - Encoder vs. Decoder Transformers

Q10a (4 points) - Differences

(i) Attention Mask:

- **Decoder-only (causal):** Uses a causal mask that prevents each position from attending to future positions. The attention matrix has $-\infty$ in the upper triangle, ensuring autoregressive generation.
- **Encoder-only (bidirectional):** Uses no mask (or only padding masks), allowing each position to attend to all other positions in the sequence, both past and future.

(ii) Training Objective:

- **Decoder-only:** Trained with causal language modeling (CLM) - predicting the next token given all previous tokens: $P(x_t | x_1, \dots, x_{t-1})$
- **Encoder-only:** Typically trained with masked language modeling (MLM) - predicting randomly masked tokens given the surrounding context (both left and right): $P(x_{\text{mask}} | x_{\text{context}})$

Q10b (2 points) - Task Example

A task decoder-only models can do that encoder-only models cannot:

Open-ended text generation - Decoder-only models can generate arbitrary-length sequences autoregressively, producing one token at a time based on all previous tokens. For example, continuing a story, writing an essay, or having a conversation.

Encoder-only models like BERT cannot naturally generate text because they are trained bidirectionally and don't have a mechanism for sequential token generation. They excel at understanding and classification tasks (sentiment analysis, NER, question answering with span extraction) but cannot produce fluent, open-ended text.

Extra Credit: Nucleus Sampling (8 points)

E1 - Nucleus Sampling Implementation

I implemented nucleus sampling (top-p sampling) in the `nucleus_sampling_generate` function. The algorithm:

1. Sort tokens by probability in descending order
2. Compute cumulative probabilities
3. Find the smallest set of tokens whose cumulative probability exceeds p
4. Zero out probabilities for tokens outside this "nucleus"
5. Renormalize and sample

Nucleus Sampling Generations (p=0.9) with Multi-Head Pile Model:

1. **Prompt:** "The quick brown fox"
Nucleus: "The quick brown fox Jump of and by on one from will with.Com- the in, about or an In in to be that it the a"
2. **Prompt:** "In the beginning"
Nucleus: "In the beginning that the the in a the of the to the to a of to and the of to the of a the the of the the"
3. **Prompt:** "What is 5 + 5?"
Nucleus: "What is 5 + 5? The of the the a the in the the a the in the in a to the of the the in the the in the"
4. **Prompt:** "The meaning of life is"
Nucleus: "The meaning of life is the the of, the in the the the a in of the for the the the the of the of the in the the"

Comparison with Temperature Sampling (from Q4):

Prompt	Temperature Sampling	Nucleus Sampling
"The quick brown fox"	Contains varied but incoherent words	More repetitive, focuses on common words
"What is 5 + 5?"	Random punctuation and fragments	Heavy repetition of "the", "a", "of"

Observations:

- **Temperature sampling** produces more diverse but chaotic outputs with random punctuation and varied vocabulary.
- **Nucleus sampling** with $p=0.9$ tends to produce more repetitive outputs, frequently sampling high-probability tokens like "the", "a", "of", "in". This is because nucleus sampling restricts the candidate set to the most probable tokens, and for a poorly trained model, these common words dominate the probability distribution.
- For a well-trained model, nucleus sampling typically produces more coherent text than temperature sampling because it avoids sampling from the very low-probability tail. However, for our small undertrained model, both methods produce incoherent outputs.

Summary of Results

Model	Dataset	Dev Perplexity
Single-Head Transformer	Pile	2479.74
Multi-Head Transformer	Pile	1318.69
Multi-Head (zero-shot)	Shakespeare	16.36
Multi-Head (from scratch)	Shakespeare	8.21
Multi-Head (fine-tuned)	Shakespeare	7.45
Multi-Head (fine-tuned)	Pile	11083.86
GPT-2 (fine-tuned)	Shakespeare	85.64